

SHOPIMPACT

Code, data and app functions

A short overview followed by a complete file-by-file code guide.

Project	This guide covers
Python Year 1 / Scenario 1	Conscious shopping, spending and rough CO2e estimates
Current source: 3 October 2026	Every function, class and main application/setup block in 20 Python files
8 categories / 5 EPA sector rows	Recorded factors, reference exchange rate, purchase fields and database tables
78 automated tests	Purpose of every test, helper and configuration file

What the app does

ShopImpact is a personal shopping journal with separate accounts. A user records product type, price, brand/shop and date. The app saves those entries, calculates a price-based CO2e estimate, shows monthly totals and goals, rewards reuse/repair habits, and suggests options to explore. Insights adds six-month trends; history supports filters, editing, deletion and backups.

How to read it

Read sections 1-3 for the brief explanation. Sections 4-12 explain the actual code at function and block level, with source line references. Repeated import, form, loop and error-handling patterns are explained once and then applied to each file. This is not a verbatim source-code printout; use the references beside each explanation to find the original lines in VS Code.

The line references describe this source snapshot and can change after later edits. Generated files, installed packages, personal databases and private credential files are outside the authored-code explanation. No real password, database URI, recovery code or private account record is included.

Current status

The app runs locally with configured Neon PostgreSQL storage. Account access and core features have been checked. Gmail inbox delivery, public Streamlit deployment, genuine participant feedback and the final assessed submission remain pending. This explanatory PDF is separate from the final submission document.

Contents

1. App functions and user journey	3
2. Data, sources and calculations	4
3. Records, database and Python building blocks	5
4. app.py - every main code block	7
5. logic.py, insights.py and storage.py	8
6. accounts.py - storage and account security	10
7. account_ui.py - login and recovery screens	12
8. email_delivery.py - optional email service	13
9. ui.py - HTML and CSS presentation	14
10. Setup tools and Turtle artwork	15
11. Tests - explanation of every check	17
12. Configuration, dependencies and references	23

1. App functions and user journey

Function	What happens
Create an account	Email, username and matching 12-128 character passwords create an empty account. Signup logs in immediately and shows a recovery-code panel on home.
Sign in / recover	Use email or username with a password. A saved recovery code can reset it; verified-email reset links become available after SMTP setup.
Record a purchase	Choose one of eight categories, enter INR price, brand/shop and date, see an instant estimate, then save. The dashboard switches to the purchase's month.
Monthly dashboard	Shows total logged spending, estimated kg CO2e and reuse/repair choices. Category chart, progress bars and achievements reflect the selected month.
Insights	Six calendar months of logged totals; average purchase, goal balance, previous-month change and largest spending category.
Explore alternatives	Three category-specific shopping approaches, selected brand/service links and a price-based comparison. No ethical certification or verified reduction is promised.
History and backups	Search, filter by month/category, sort, download filtered CSV, edit/recalculate or confirm deletion. Full JSON backups restore records; CSV reports open in spreadsheets.
Preferences / ownership	Save monthly goals and high contrast, reload after a conflicting edit, sign out, or confirm password to delete the account.

The main execution path

Browser input

- > app.py / account_ui.py (interface + session state)
- > logic.py / insights.py / storage.py (Python rules)
- > accounts.py (authenticated, revision-checked save)
- > SQLite locally OR Neon PostgreSQL when configured
- > rerun -> refreshed totals, charts, history and rewards

Streamlit reruns app.py after interactions. `st.session_state` retains the current session token, cached account data and widget state during that browser session. The database keeps purchases and goals after a restart. A page refresh currently requires another sign-in. The interface's temporary memory is therefore different from permanent account storage.

2. Data, sources and calculations

Data files used by the app

File	Contents and use
data/product_catalog.json	Eight category dictionaries: multiplier, lower_impact habit flag, three alternatives, compare_with, EPA factor/code/title and mapping note. Loaded by logic.py.
data/emission_sources.json	EPA dataset/version, five recorded source rows, chosen factor column, historical currency rate, URLs, retrieval date and SHA-256 file fingerprint. Evidence of factor provenance; not a live feed.
data/alternative_links.json	Category-indexed official links and plain-language notes for Patagonia Worn Wear/repairs, VEJA repair, Cashify repair/refurbishment/warranty and Klean Kanteen bottles.
data/eco_tips.json	Five authored suggestions about using existing items, repair, borrowing, product information and personal progress. The tip index rotates through them.
tests/sample_purchases.json	15 synthetic varied purchases with the current method version. Test/example evidence only, never automatic live-account content.
assets/turtle/leaf.svg	Generated vector reward from tools/draw_rewards.py. Displayed when the selected month includes a flagged choice.

Source-backed reference model

The selected EPA v1.3.0 data describes US supply-chain factors by industry, expressed in kg CO₂e per 2022 USD. The app uses the **with margins** column, then divides by its recorded 2022 reference rate of **78.6044905829916 INR per USD**. This gives a category multiplier per INR. Source links: EPA dataset catalogue and the World Bank indicator request retained in the project.

```
factor_per_INR = EPA_factor_per_2022_USD / reference_INR_per_USD
estimated_CO2e = purchase_price_INR * factor_per_INR
result = round_to_0.01_kg_using_Decimal_and_HALF_UP
```

Category	NAICS	kg/USD	Stored kg/INR
New clothing	315990	0.12	0.001526630338
Second-hand clothing	315990	0.12	0.001526630338
Leather footwear	316210	0.282	0.003587581293
New electronics	334111	0.058	0.000737871330
Refurbished electronics	334111	0.058	0.000737871330
Single-use household goods	326160	0.579	0.007365991379
Reusable household goods	326160	0.579	0.007365991379
Repair services	811490	0.113	0.001437576901

CO2e means carbon dioxide equivalent: a common unit for greenhouse gases. NAICS is the industry classification used to choose a sector. The exact stored multipliers above, not rounded display labels, drive the calculation.

Worked example

Second-hand clothing, INR 500
 $500.00 * 0.001526630338 = 0.763315169000$
Displayed estimate: 0.76 kg CO2e

At the same price, new clothing receives the same factor. The reuse badge counts the type of choice; it does not invent an emissions discount. Similarly, refurbished/new electronics share a computer proxy, while reusable/single-use household goods share a plastic-bottle proxy.

Important limits

These are US sector averages converted at a fixed historical rate. Current INR prices are not inflation-adjusted. Footwear is not leather-specific, electronics uses a computer sector, and household goods uses a broad bottle proxy. A free purchase returns zero in this price model, which does not mean zero physical impact. Lower price can lower the estimate without lowering actual emissions. Dates can include future/planned purchases. Brand links document options, not independent sustainability certification.

The app makes no external EPA or exchange-rate request on each purchase; it reads its bundled JSON. The source file fingerprint recorded in `emission_sources.json` is `6025a14c4fe16675735efba1683030e4d36011e3973f3c4261f8231aa69c002f`.

3. Records, database and Python building blocks

One purchase is a dictionary; purchases form a list

Key	Meaning / representation
id	Unique UUID string; edits preserve it. Synthetic tests use labelled demo IDs.
product_type	One exact category key from CATALOG.
price	Validated INR money stored as a two-decimal string.
brand	Trimmed brand/shop text, 1-80 characters.
purchase_date	ISO YYYY-MM-DD, from 2000-01-01 to 2100-12-31.
multiplier	Recorded decimal-string factor per INR.
estimated_co2	Rounded decimal-string estimated kg CO2e.
methodology	epa-2022-inr-reference-v2 identifies the calculation method.
source_code	Six-digit EPA NAICS source code for that category.

Permanent SQL tables

Table	Fields and purpose
users	id, unique username, unique email index, email_verified, password_hash, recovery_hash, purchases JSON text, settings JSON text and revision.
sessions	token_hash, user_id and expires_at; links an authenticated random token to its account for up to 12 hours.
attempts	name, failures and blocked_until; shared email/username login counter and separate recovery counter.
email_actions	token_hash, user_id, purpose, email, password_snapshot and expires_at; binds single-use email actions to unchanged account details.
email_requests	address_hash and next_allowed; limits resend frequency without keeping an extra plaintext address here.

Default settings are budget=10000.0, carbon_goal=100 and contrast=False. SQLAlchemy transactions use bound values rather than concatenating user input. Neon hosts PostgreSQL; it does not supply the app's login system. ShopImpact's Accounts class implements that system. The administrator can access stored purchase data: account isolation is not end-to-end encryption.

Python patterns you should understand

Pattern	Why it appears
import / constants / docstrings	Imports expose library helpers; ROOT, CATALOG and VERSION define shared configuration. Docstrings/comments explain intent and do not perform a shopping calculation.
Functions, classes and return	Functions separate reusable calculations; Accounts and Mailer keep related operations together. Return values carry results back to callers.
Lists, dictionaries, sets	Lists hold purchases/tips/results. Dictionaries hold fields/configuration. Sets collect unique months/IDs and make duplicate checks easy.
Comprehensions, loops, conditions	Filter and rebuild records, group category totals, generate drawing steps and choose valid actions. any(), sum(), max() and sorted() express common summaries.
Decimal and datetime	Decimal keeps calculation inputs exact; date parses valid dates and ISO strings group entries by year/month.
try/except/finally and with	Translate expected errors into clear messages; close database/SMTP connections and always release background queue slots.
JSON, CSV, Path and f-strings	Exchange portable records, resolve project files and format totals without changing the stored Decimal values.
Decorators / dataclasses / __main__	Cache a service, define structured email settings, or run a tool/test only when launched directly.

4. app.py - every main code block

app.py has no user-defined functions. Its module-level blocks compose the actual shopping screen. It imports shared functions instead of duplicating business rules. The following map covers the full file in execution order.

Imports and startup | lines 2-23

Imports JSON/date, Streamlit and project services. `set_page_config` chooses the title, icon and wide layout. Styling is applied before the account gate. `require_account` returns the service and authenticated data or stops at sign-in. The tip counter starts once per session; settings and records reference loaded account content, and a fixed key namespaces download/import controls.

Sidebar: account, reload and preferences | lines 24-74

Shows the username, current email and optional verification action. The email-change form confirms the current password, reloads the user and can queue new/old-address notices. `AccountError` and SQL errors become messages. Sign-out revokes the token. Reload drops cached account data and goal widget state, then reruns. The goals form saves budget, CO2e goal and contrast together; it refreshes the local settings reference on success so Insights immediately sees the new goal.

Header, recovery panel and legacy session import | lines 75-103

Applies the selected palette, adds the title/estimate notice and shows signup success. A new account can expand, download and dismiss its private recovery code. A previous-session list, if present, can be explicitly imported: each entry is rebuilt using current factors, duplicate IDs are skipped, saving is confirmed, and the old session list is removed. Four tabs organise the shopping screens.

Dashboard period and cards | lines 105-123

Builds a descending set of current and recorded months. A successful purchase can set `next_dashboard_month`; an invalid old month selection is cleared. `month_records` and `summary` produce three cards: INR spend, estimated kg CO2e and flagged-choice count. A one-time `purchase_saved` message confirms the refreshed history.

Live entry and confirmed save | lines 124-144

Four labelled controls collect category, price, shop and date. `impact` calculates the preview before saving. The Add button checks the 1,000-record limit and creates a new dictionary. `save_account` must succeed before the tip counter, saved flag and next month change. `st.rerun` then rebuilds the screen from confirmed data; invalid values show `ValueError` messages.

Category chart, goals, badges and Turtle | lines 145-184

A radio selects estimated impact or spending. A loop groups the selected month's entries by category and draws a green bar chart. Empty months show guidance. Progress is capped at 100% for display while text preserves actual usage and over-goal notices. `badges` decides achievements; a flagged choice conditionally displays the generated leaf SVG. Fixed rules are explained in an expander. Tips are loaded from JSON and selected with index modulo list length.

Suggestions and comparison | lines 185-203

Reads the selected category's three alternatives and mapping note, then loads its named options from `alternative_links.json`. The comparison expander lets the user choose another category/price, runs `impact` for both and shows their estimated difference. Captions explain that a smaller price-based result is not measured emissions savings.

Six-month Insights | lines 204-224

The selected period feeds `month_insights` using the current saved budget. Three cards show average price, remaining/exceeded spending goal and previous-month change. `monthly_trend` supplies a green line chart for either spending or estimated impact. A message names the largest spending category, and an expandable table exposes the six months' values. Missing baseline/entries are labelled explicitly.

Filtered history, edit and delete | lines 225-263

Search plus month/category controls and sorting feed `filter_purchases`. Non-empty results show their count, totals, table and filtered CSV. The edit ID lookup uses only visible results, so hidden purchases cannot accidentally be selected. Saving calls `make_purchase` with the original ID to recalculate fields, then `update_purchase`. Deletion needs an explicit checkbox. A zero-result view hides edit/delete controls without removing data; an empty account gets guidance.

Full backups and restoration | lines 264-279

Separate buttons export all purchases as JSON or CSV, regardless of history filters. The uploader accepts JSON; replacement requires confirmation. A separate checkbox allows old illustrative versions. `import_json` validates/recalculates the upload, `save_account` replaces the records only after validation and a successful revision check, and the UI reruns.

Methodology, privacy and deletion | lines 280-304

Explains sources, fixed exchange rate, broad category mappings, model limitations, badges, account storage, dates and Turtle integration. A dataframe lists the factor mappings. Account deletion is inside an expander/form and requires password plus consent; success deletes the user's database data, clears browser state and reruns. Expected failures are caught and displayed.

5. logic.py, insights.py and storage.py

logic.py - validated calculations

Imports JSON, date, Decimal helpers, Path and UUID. `ROOT` resolves the project folder; `CATALOG` loads `product_catalog.json` at import; `VERSION` identifies the current methodology. These modules have no Streamlit interface dependencies, which makes their rules straightforward to test.

money | lines 13-21

Converts the input to Decimal through `str()`, rejects non-finite values and prices outside INR 0-10,000,000, and rounds to two decimal places using `ROUND_HALF_UP`. Invalid decimal input becomes a friendly `ValueError`. Decimal avoids ordinary floating-point rounding surprises.

impact | lines 24-28

Checks that the category exists, validates the price through `money()`, multiplies it by the category's recorded Decimal multiplier, and rounds the result to 0.01 kg CO₂e. It returns a Decimal, not formatted text.

make_purchase | lines 31-46

Validates a trimmed brand/shop name of 1-80 characters and parses an ISO date between 2000 and 2100. It calculates impact, creates or preserves an ID, and returns a dictionary containing entered and derived fields. Money and factors are stored as decimal strings for exact JSON round trips.

month_records | lines 49-50

A list comprehension returns purchases whose ISO date begins with the selected YYYY-MM. This separates different years as well as months and does not modify the input list.

summary | lines 53-57

Adds price and estimated_co2 strings as Decimals, counts purchases, and counts categories with the lower_impact flag. Returns spend, co2, count and lower. Empty histories have zero totals. The flag rewards a type of shopping choice; it does not prove a measured emissions reduction.

badges | lines 60-70

Applies fixed monthly rules: Eco Saver requires at least three entries and at most 50 kg estimated CO2e; Low Impact Shopper requires at least three flagged choices and at least 60% flagged entries; Repair Champion requires a Repair services entry. Short-circuit evaluation prevents division by zero for an empty month.

update_purchase | lines 73-76

Checks that the replacement ID exists, then returns a new list replacing only the matching record. Keeping the same ID lets edited price, date, category and brand stay attached to the original purchase. Missing IDs raise ValueError.

demo_records | lines 79-87

Creates 15 deliberately fictional purchases across a given/current month and the preceding month. It cycles through the eight categories, uses varied fixed prices and demo IDs, and handles January's year boundary. This is a test fixture only; the real app does not call it to populate accounts.

insights.py - trends and history views

Imports Decimal rounding plus month_records/summary. All results come from the logged list; it does not fetch bank transactions, predict shopping or measure physical carbon savings.

shift_month | lines 6-11

Splits YYYY-MM into integers, checks the month, converts the date to a continuous month index, adds the offset, and uses divmod to reconstruct year and month. This handles negative offsets and December/January boundaries without assuming a fixed number of days.

monthly_trend | lines 14-17

Builds six consecutive calendar months ending at the selected month. Each dictionary merges the period with summary(month_records(...)). Empty periods stay in the trend as zero logged totals rather than disappearing.

month_insights | lines 20-32

Summarises the selected month and previous calendar month, accumulates Decimal spending by category, and selects the largest spending category. It returns average purchase rounded to cents, budget minus spending, and absolute spending change. A missing previous month produces None; a previous month with a free purchase is a valid zero-spend baseline. Current saved goals are used, not historical goal snapshots.

filter_purchases | lines 35-47

Normalises search with strip/casefold, combines optional month and category conditions, and searches brand, category, date, price and estimate. Returns a fresh sorted list. Price and impact sorts use Decimal values; date sorts use ISO strings. Date and ID break ties consistently. The original saved list is untouched.

storage.py - portable purchase files

Uses csv/io/json and the shared VERSION/make_purchase rule. Backup validation is independent of database persistence; loading a file does not itself write an account until app.py confirms the save.

export_json | lines 8-9

Serialises a wrapper containing methodology version, INR currency and the purchases list using indented JSON. It excludes passwords, sessions, recovery codes and goal settings.

import_json | lines 12-39

Rejects inputs over 2,000,000 bytes, malformed JSON, unexpected wrapper/currency/version, more than 1,000 records and duplicate or invalid IDs. Legacy illustrative backups require allow_legacy=True. Every row is rebuilt through make_purchase, so uploaded impact/factor values cannot override the calculation. Missing fields receive a friendly error.

export_csv | lines 42-52

Writes a header and selected purchase fields into an in-memory StringIO buffer using DictWriter. IDs and unrelated extra fields are omitted. Values beginning, after whitespace, with =, +, - or @ get an apostrophe prefix to prevent spreadsheet formula interpretation. Returns CSV text for download.

6. accounts.py - storage and account security

Imports hashing, JSON, regex, randomness, time and Path, then Argon2 and SQLAlchemy. HASHER uses time_cost=2, memory_cost=19456 and parallelism=1. A random DUMMY_HASH allows an unknown login name to run a password check too. DEFAULT_SETTINGS supplies a new account's goals and appearance. Exact values of real private settings are intentionally omitted.

AccountError | lines 19-20

A ValueError subclass for expected account failures. The interface catches this class and shows a normal user message rather than an unhandled traceback.

ConflictError | lines 23-24

An AccountError subclass indicating that another browser tab saved a newer revision. It prompts the user to reload instead of silently overwriting those changes.

digest | lines 27-28

Encodes a token as bytes and returns its SHA-256 hex digest. The database stores digests for unpredictable session/recovery/email tokens. User passwords instead use Argon2id, which is intentionally slower to guess.

username_key | lines 31-34

Requires 3-40 ASCII letters, digits, dots, underscores or hyphens, trims whitespace and lowercases the name. The regular expression defines which usernames are valid and creates one consistent lookup key.

check_password | lines 37-39

Requires a string with 12-128 characters. It checks length; it does not demand a special-character pattern or check compromised-password lists.

email_key | lines 42-52

Checks an ASCII email pattern, total length up to 254, local-part length up to 64, and disallows leading/trailing or repeated dots. It trims and lowercases the address. This is syntax validation, not proof that a mailbox exists.

Accounts | lines 55-307

Owns the SQLAlchemy engine and all account operations. Methods execute parameterised SQL inside connection or transaction context managers. A single user row stores purchases and preferences as JSON text, while other tables store sessions and recovery-related controls.

Accounts.__init__ | lines 56-99

Stores the optional verification policy and rejects SQLite or a missing URL in public mode. Locally it creates personal_data/shopimpact.db when no URL is supplied. PostgreSQL URLs select the psycopg driver; SQLite allows cross-thread access with a 15-second timeout. Startup creates five tables, safely adds older accounts' missing email columns, and adds a unique email index. SQLite BEGIN IMMEDIATE serialises its migrations; pool_pre_ping detects stale pooled connections.

Accounts.register | lines 101-116

Normalises the username/email, validates password length, creates random user/recovery IDs, hashes the password and recovery code, and inserts an empty purchases list plus default settings. Duplicate username/email errors are translated into AccountError. Returns the raw recovery code once; the database keeps its digest.

Accounts._identity | lines 118-125

Chooses email lookup if the supplied identifier contains @, otherwise username lookup. The interpolated SQL field is selected only from these two fixed names, while the value is bound as a parameter. Returns both the user row and its canonical username so email and username aliases share login lockout.

Accounts._blocked | lines 127-131

Reads an identifier's blocked_until timestamp. A future timestamp produces the normal 15-minute lockout message; missing or expired entries allow checking to continue.

Accounts._failure | lines 133-143

Uses INSERT ON CONFLICT and atomic UPDATE statements to increment failures without losing parallel attempts. An expired lockout starts a new count. At five failures it sets blocked_until to current time plus 900 seconds.

Accounts.login | lines 145-170

Finds the identity, checks lockout, and verifies the password using Argon2. Unknown accounts still perform a dummy-hash verification. Incorrect input increments the shared counter. Optional service-level email verification can gate login, but the app passes False. Success removes failures and expired sessions, then inserts a random 12-hour session only if the password snapshot still matches. This prevents a concurrent reset from accepting old credentials.

Accounts._user | lines 172-178

Joins users to sessions by the supplied token's digest and checks expires_at. It returns only the authenticated user's row. A missing or expired session raises AccountError.

Accounts.load | lines 180-184

Uses _user(), decodes purchases/settings JSON and returns the username, email, verification flag and revision. The raw password/recovery hashes and session data are not returned to the shopping interface.

Accounts.save | lines 186-197

Rejects more than 1,000 purchases, validates the session, JSON-encodes the chosen purchases/settings, and updates only the authenticated user at the expected revision. Exactly one updated row advances revision by one; otherwise ConflictError prevents a stale tab from overwriting new data.

Accounts.logout | lines 199-201

Deletes the matching hashed session token inside a transaction. This revokes that session; the UI separately clears its in-memory state.

Accounts.reset_password | lines 203-219

Validates a replacement password and the recovery code using compare_digest, with a separate recovery lockout counter. A conditional update allows the old code to be used once, hashes the new password, and rotates recovery. It deletes all user sessions, email actions and relevant failure counters before returning a replacement code.

Accounts.set_email | lines 221-235

Validates the new address, authenticates the session and confirms the current password. A changed address resets email_verified and deletes outstanding email actions. Duplicate addresses become a friendly error. Returning the normalised email lets the interface refresh the user's data.

Accounts.request_email_action | lines 237-261

Allows only verify/reset, normalises the address and atomically applies a 60-second per-address cooldown using a hashed address key. It silently returns None for unknown or ineligible addresses. Eligible requests invalidate previous same-purpose tokens and store a hashed token bound to user, email, password snapshot and expiry: 24 hours for verification, 30 minutes for reset. Only internal delivery code receives the raw token.

Accounts.consume_email_action | lines 263-296

Checks purpose/token shape and validates/hashes a replacement password when resetting. DELETE RETURNING atomically claims an unexpired single-use token. A conditional user update must match the original email/password snapshot. Verification marks the address verified; reset rotates password/recovery and revokes sessions. It removes remaining email actions, resets lockout on reset, and returns email plus the new recovery code when relevant.

Accounts.delete_account | lines 298-307

Validates the session and confirms the password, then deletes the user's sessions, email actions and user row in one transaction. Purchase/settings JSON disappears with the user row. Downloaded backups on the user's computer are not deleted.

Why password hashes differ from token digests

An ordinary password may be guessable, so it uses salted, memory-intensive Argon2id. Session, recovery and email tokens are generated with secrets and have high entropy, so a fast SHA-256 lookup digest is appropriate. Hashing is not encryption: the app verifies supplied values rather than decrypting stored ones. SQL transactions and conditional updates protect both single-use actions and changes made from multiple tabs.

7. account_ui.py - login and recovery screens

Imports environment/configuration access, Streamlit and the expected account/database/secret exceptions. EMAIL_REQUEST_MESSAGE is deliberately generic. account_service.clear aliases the cached resource's clear method for tests. At runtime the current app calls account_service(..., False), so email verification is optional for dashboard access.

_cached_account_service | lines 12-13

@st.cache_resource keeps the account service/connection engine available across Streamlit reruns. The URL, public flag, verification policy and implementation identity form the cache arguments.

account_service | lines 16-19

Passes id(Accounts) into the cached wrapper. When development reloads the class, a fresh service is created instead of retaining an instance with stale exception classes. The module's clear alias lets tests dispose/reset this cache.

email_settings | lines 25-39

Collects eight email configuration keys, preferring values found in os.environ and otherwise trying st.secrets. A missing secrets file is tolerated. It reads private values only at runtime; none belongs in source control or this document.

configured_mailer | lines 42-48

Returns None when email is disabled, including an early environment false check. When enabled it converts the gathered settings into a Mailer. The app can operate with recovery codes while SMTP is unconfigured.

email_link_screen | lines 54-109

Moves verification/reset query parameters into session state, checks their shape and clears the URL to reduce token exposure. Confirmation consumes a verify token; a reset form checks matching new passwords, consumes the token, clears sessions and shows the replacement recovery code. Invalid links and database failures become messages. A return button clears link state. st.stop prevents the shopping page from rendering during this flow.

request_email_form | lines 112-122

Builds a purpose-specific form. Submission is disabled without a mailer. Valid input queues a background request and receives the same generic response whether an eligible account exists or not.

sign_out | lines 125-129

Revokes the current database session if present, clears all session state and reruns into the sign-in page.

require_account | lines 132-243

Loads database/public/email configuration and catches expected setup errors. It first handles email links, then validates and loads an existing authenticated session. If no valid token exists, it draws the split sign-in/create/recover page and stops the app. Signup checks confirmation, registers, immediately logs in and reruns to home with a recovery-code panel; optional email delivery does not gate access. Sign-in accepts email or username. Recovery uses a verified-address email link when configured or the saved recovery code.

save_account | lines 246-258

Defaults omitted purchase/settings arguments to the current account data, calls service.save with the session token and expected revision, and catches conflicts/database failures. Only after success does it update the in-memory account dictionary and revision. Returns True/False so caller screens rerun only after a confirmed save.

Three entry tabs, immediate home access

The unauthenticated screen has Sign in, Create account and Recover account. It has no Verify email tab. After registration the app stores the token, clears stale account data and reruns to home; the recovery code is presented there. Optional verification remains in the signed-in sidebar and is needed for email reset eligibility. The code still supports a stricter reusable Accounts policy for tests or another caller, but that is not the current UI policy.

Configuration precedence caveat

The helper checks environment values before reading missing private-file values. Streamlit itself can also copy top-level secret values into the environment during startup. Consequently an isolated browser preview must use its own secrets-file path; merely setting environment values is not a universal guarantee that a different Streamlit secrets file will never be loaded. The test suite uses temporary accounts and explicit test configuration.

8. email_delivery.py - optional email service

Imports dataclasses, EmailMessage, SMTP/SSL, a thread pool, a bounded semaphore and URL helpers. Two background workers and eight slots limit concurrent/queued work. The pool exists at module level across ordinary calls. This code prepares verification/reset links and account notices; real Gmail sender setup is still pending.

EmailConfigurationError | lines 13-14

A ValueError subclass for invalid sender, link or transport configuration. It distinguishes setup failures from a user's ordinary account error.

Mailer | lines 18-81

A dataclass holding SMTP host/port, username, password, sender, app URL and security mode. repr=False keeps its settings out of ordinary object representations, and the password field is also excluded from representation.

Mailer.__post_init__ | lines 27-37

Normalises the sender email and checks the app URL. Public links require HTTPS; localhost may use HTTP. Credentials, query strings and fragments in the base URL are rejected. It validates the host, required login settings, SSL/STARTTLS mode and port range, and strips a trailing slash.

Mailer._connect | lines 39-57

Creates a certificate-verifying SSL context. SMTP_SSL encrypts from the start; STARTTLS performs EHLO, upgrades the connection, then repeats EHLO before login. Both use a 15-second timeout. Failure closes the connection before raising; successful authentication returns it.

Mailer.check_login | lines 59-61

Opens and closes an authenticated SMTP connection without sending a message. Used by transport tests as a minimal login check.

Mailer.send_message | lines 63-71

Validates the recipient, constructs a plain-text EmailMessage with From/To/Subject, opens the secure connection and submits it to the provider. Provider acceptance is not a guarantee of inbox delivery.

Mailer.send_action | lines 73-81

Builds the configured app URL with URL-encoded email_action/email_token parameters. Chooses the verification or reset title and expiry notice, includes single-use instructions and sends through send_message. It rejects unknown purposes.

mailer_from_settings | lines 84-91

Converts private configuration values to a Mailer, defaulting the provider to smtp.gmail.com, port 465 and SSL. Sender address, password and app URL must be supplied privately for sending to work.

queue_email_action | lines 98-117

Validates the address and acquires one of eight queue slots without waiting. It submits a nested delivery task to a two-worker thread pool and releases the slot if submission fails. The browser is not given a secret token or an account-existence answer.

queue_email_action.deliver | lines 104-113

Requests the eligible token payload from the account service and sends only if a payload exists. It suppresses errors to avoid exposing credentials, addresses or tokens, and always releases the reserved semaphore slot. This in-memory queue has no durable retry or delivery-status tracking.

queue_notice | lines 120-133

Skips missing email or a full queue, then submits a plain account-change notice with the same bounded-worker pattern. A submit failure releases the slot.

queue_notice.deliver | lines 123-129

Sends the supplied plain notice, suppresses provider failures and releases the queue slot in finally. Notices do not create verification/reset tokens.

Service boundary

Gmail sends messages, Neon stores accounts, and the Streamlit app validates tokens and passwords. They have separate roles. Gmail integration here is SMTP with an app password, not Google OAuth or 'Sign in with Google'. Background submission deliberately does not prove delivery; exceptions are suppressed and queued tasks are not durable across process shutdown.

9. ui.py - HTML and CSS presentation

Imports only Streamlit. The interface retains native labelled widgets, while a static stylesheet changes their visual presentation. `unsafe_allow_html=True` is used for authored CSS/brand markup, not to insert user-entered brand names as raw HTML.

apply_styles | lines 5-52

Selects normal/high-contrast background, sidebar and muted text colours, then inserts static scoped CSS. Variables keep colours consistent. Heading clamp sizes, a 1,380px maximum content width, metric cards, rounded forms/controls, tab spacing and focus outlines provide hierarchy. Captions keep opacity 1. The media query at 760px changes padding, stacked-row gaps, card sizes and brand details for mobile. Double braces escape CSS braces inside the Python f-string. Native Streamlit controls keep their labels and behaviour.

brand_panel | lines 55-62

Inserts a fixed HTML introduction panel with the ShopImpact wordmark, short explanation and a decorative inline SVG leaf. This is a separate visual from the genuine Turtle reward. The markup contains no user-entered text and no remote fonts/images.

CSS group	Effect
.stApp + custom properties	Shared cream/white surfaces, green text, muted captions and borders; contrast-dependent colours.
Headings / main block	Responsive h1 clamp, tighter letter spacing, max content width and consistent padding.

CSS group	Effect
Captions / sidebar	Opacity 1 preserves readable caption colours; pale sidebar and a separating border.
Metrics / forms / expanders	White rounded cards, borders, small shadow, useful spacing and clear grouped forms.
Buttons / focus / input / tabs	Dark green primary buttons with white text, visible focus outline, rounded input surfaces and spaced tab labels.
Brand panel / eyebrow / SVG	Editorial introduction panel and decorative leaf; small uppercase journal label.
760px media query	Mobile padding clears the app header, stacked-row spacing and smaller panels/cards; secondary brand details are hidden.

CSS data-testid selectors depend on Streamlit's generated markup. A future Streamlit upgrade may require revisiting them. The high-contrast toggle and viewport checks are useful improvements, but full keyboard/screen-reader assessment has not been completed.

10. Setup tools and Turtle artwork

tools/connect_database.py

Imports hidden input, arguments, file/system/process helpers, TOML and SQLAlchemy. ROOT and sys.path make the project's Accounts module available when launching the tool directly. Its `__main__` guard adds `--clipboard` and exits with main's success/failure code.

validate_url | lines 17-25

Parses the supplied URI using SQLAlchemy `make_url`. It accepts PostgreSQL drivers only, requires `host/database/username/password`, and requires `sslmode=require` or stronger. It selects the `psycopg` driver and retains the provider's other URI parameters.

read_clipboard | lines 28-38

On Windows, runs a hidden non-interactive PowerShell clipboard read, capturing text in memory with a 10-second timeout. Credentials are not placed in process arguments or printed. Other operating systems receive a setup error.

main | lines 41-60

Prompts for a hidden URI through `getpass`, or waits for copied input in clipboard mode. It validates the URI, avoids printing raw parsing failures and calls `configure_database`. Return code 0 means success and 1 means failure.

connection_error_message | lines 63-81

Examines provider exceptions internally for authentication, hostname, timeout, refusal or TLS markers, then returns a fixed safe explanation. It never returns the original provider text, which might contain a password or host details.

configure_database | lines 84-119

Tests `SELECT 1` with a connection timeout, initialises the account schema in public PostgreSQL mode and disposes temporary engines. Only after successful connection does it merge settings and write a private temporary TOML file, replacing `secrets.toml` atomically with `os.replace`. Failure removes a temporary file and leaves previous settings unchanged. Local accounts are not automatically copied to Neon.

tools/database_setup_ui.py - module-level blocks

Imports output-capture/path helpers and Streamlit, then adds the project root to sys.path and imports the same validation/configuration functions. It sets a centred setup page and checks that server.address is a loopback address; otherwise it displays an error and stops. A password-masked URI form clears on submit. On connection, validation errors are hidden; successful validation captures credential-free setup messages with redirect_stdout while a spinner runs. A successful result shows a link to the app, and failure displays the sanitised helper message. It is an owner-only local setup page, not a public app feature.

tools/email_setup_ui.py - module-level blocks

Imports file/atomic-write helpers, Streamlit/TOML, email_key and Mailer. The same loopback-only check protects the setup page. The form collects sender, masked Google app password and app URL; a submit removes spaces from the app password, validates the sender, sends a test message to that sender over SMTP SSL port 465 and only then merges private settings. A temporary file plus os.replace preserves the existing file on failure; temporary files are removed. Success means SMTP accepted the test, not guaranteed inbox delivery. The __main__-style script body is rendered by Streamlit rather than exposing a separate function.

One setup-page information string still describes verification as required after setup. The actual app policy is determined by require_account(), which currently keeps verification optional for home access. Use the executable policy explained in section 7 when describing the product.

tools/draw_rewards.py - genuine Turtle navigation

Imports argparse, Path and the Python standard-library turtle module. The generated SVG is committed/displayed by the app so cloud rendering does not need a live Tk window. Desktop --preview requires working Tk and remains unverified in the bundled runtime.

SvgTurtle | lines 12-59

Subclasses Python's turtle.TNavigator to record navigation in SVG paths instead of requiring a Tk window on a hosted server. It is a small drawing adapter, not a replacement for the Turtle navigation engine.

SvgTurtle.__init__ | lines 15-21

Initialises Turtle navigation, a pen-down flag, path buffers, dark green colour and starting line width.

SvgTurtle._goto | lines 23-28

Records the current and destination coordinates when drawing is enabled, then calls the inherited _goto to maintain the real Turtle position. Pen-up movement changes position without recording a line.

SvgTurtle.flush | lines 30-33

Stores the current point sequence with its current colour/width and clears that sequence. Empty paths are ignored.

SvgTurtle.penup | lines 35-37

Flushes existing lines and turns drawing off so the pen can move without joining unrelated parts.

SvgTurtle.pendown | lines 39-40

Turns recording on so later Turtle movements produce a visible path.

SvgTurtle.pencolor | lines 42-44

Flushes the previous path before changing its colour, preserving earlier strokes' styling.

SvgTurtle.pensize | lines 46-48

Flushes the previous path before changing its line width.

SvgTurtle.svg | lines 50-59

Flushes remaining lines, builds the SVG header/title/background, writes a polyline for each recorded path and joins the fragments into text. Negating y converts Turtle's upward-positive coordinates to SVG's downward-positive coordinates. Rounded line caps/joints give the leaf a smooth outline.

draw_leaf | lines 62-88

Uses repeated forward steps and left turns to draw two curved sides, then draws the diagonal central vein and pairs of smaller veins. The same recipe accepts either `SvgTurtle` or a desktop `Turtle` instance.

main | lines 91-109

Reads `--preview`, draws and writes `assets/turtle/leaf.svg`, creating its folder when needed. Optional preview opens a Turtle screen with matching colours, runs the recipe and starts the Tk event loop. The `__main__` guard runs this only when the tool is launched directly.

11. Tests - explanation of every check

The 78 tests were passing in the project's recorded 3 October verification. They use `unittest`, temporary SQLite databases, mocks and `Streamlit AppTest`. SMTP tests do not send real mail; setup tests do not need a live Neon connection. Below, each test purpose is derived from its actual source and identifier; source lines locate the executable assertions. These checks are evidence for the tested cases, not a guarantee of every browser, service or accessibility scenario.

tests/test_account_service.py

1 cache regression test: a reloaded Accounts subclass must receive a fresh cached service, avoiding stale exception-class behaviour.

ServiceCacheTests (lines 11-28): A `unittest.TestCase` group; assertions report success or a precise failure.

ServiceCacheTests.test_reloaded_implementation_does_not_reuse_stale_service.ReloadedAccounts (lines 16-17): A temporary subclass used to model a development reload.

Test function	Lines	Purpose
<code>test_reloaded_implementation_does_not_reuse_stale_service</code>	12-28	Checks reloaded implementation does not reuse stale service.

tests/test_accounts.py

16 database/account tests: isolation, persistence, password hashing, validation, lockout, revisions, recovery, sessions and email migration.

AccountTests (lines 13-166): A `unittest.TestCase` group; assertions report success or a precise failure.

AccountTests.setUp | lines 14-19

Creates the fixture used by each test, such as a disposable database/account or a small purchase list. This runs before each test so unrelated checks do not share state.

AccountTests.tearDown | lines 21-23

Disposes database engines, clears relevant cached services/environment patches and removes temporary files after each test. This keeps real accounts separate from test evidence.

AccountTests.test_password_reset_during_login_cannot_issue_old_session.reset_after_verification | lines 73-76

A nested test hook resets the fixture password immediately after the old password verifies, exercising the race between login and a password reset.

Test function	Lines	Purpose
<code>test_private_data_and_restart</code>	25-33	Checks private data and restart.
<code>test_passwords_are_salted_hashes</code>	35-40	Checks passwords are salted hashes.

Test function	Lines	Purpose
test_duplicate_and_weak_password	42-45	Checks duplicate and weak password.
test_failed_login_rate_limit	47-52	Checks failed login rate limit.
test_concurrent_edits_do_not_overwrite	54-58	Checks concurrent edits do not overwrite.
test_recovery_revokes_sessions_and_rotates_code	60-67	Checks recovery revokes sessions and rotates code.
test_password_reset_during_login_cannot_issue_old_session	69-82	Checks password reset during login cannot issue old session.
test_logout_and_expired_sessions	84-92	Checks logout and expired sessions.
test_delete_requires_password_and_removes_account	94-99	Checks delete requires password and removes account.
test_hosted_mode_rejects_local_database	101-103	Checks hosted mode rejects local database.
test_email_login_normalizes_case_and_preserves_username	105-110	Checks email login normalizes case and preserves username.
test_invalid_and_duplicate_emails_are_rejected	112-116	Checks invalid and duplicate emails are rejected.
test_link_email_requires_password_and_preserves_saved_data	118-134	Checks link email requires password and preserves saved data.
test_email_and_username_share_login_rate_limit	136-142	Checks email and username share login rate limit.
test_email_recovery_revokes_existing_sessions	144-150	Checks email recovery revokes existing sessions.
test_existing_database_migration_keeps_accounts_and_purchases	152-166	Checks existing database migration keeps accounts and purchases.

The final `__main__` guard calls `unittest.main()` when this file is run directly. Discovery can also load it as part of the full suite.

tests/test_app.py

12 interface tests: real app forms, immediate signup-to-home, normal login errors, add/edit/delete, goal saving, filtering and email-link flows.

AppTests (lines 19-213): A `unittest.TestCase` group; assertions report success or a precise failure.

by_label | lines 15-16

Selects the Streamlit test element with a given visible label, letting an assertion or click target the intended control.

AppTests.setUp | lines 20-27

Creates the fixture used by each test, such as a disposable database/account or a small purchase list. This runs before each test so unrelated checks do not share state.

AppTests.tearDown | lines 29-37

Disposes database engines, clears relevant cached services/environment patches and removes temporary files after each test. This keeps real accounts separate from test evidence.

AppTests.app | lines 39-47

Launches the actual Streamlit app under test, checks that no exception occurred and the sign-in gate has no metrics, then signs into the fixture account and returns the authenticated `AppTest`.

Test function	Lines	Purpose
test_initial_empty_dashboard	49-52	Checks initial empty dashboard.
test_history_filters_limit_edit_choices_and_keep_saved_data	54-67	Checks history filters limit edit choices and keep saved data.
test_insights_follow_logged_month_and_saved_goal	69-80	Checks insights follow logged month and saved goal.
test_incorrect_login_shows_message_without_crashing	82-89	Checks incorrect login shows message without crashing.
test_blank_brand_rejected	91-95	Checks blank brand rejected.
test_add_edit_delete	97-112	Checks add edit delete.
test_refresh_retains_purchases_and_settings	114-127	Checks refresh retains purchases and settings.
test_registration_then_login	129-149	Checks registration then login.
test_existing_account_can_link_email_then_sign_in	151-162	Checks existing account can link email then sign in.
test_verification_link_is_removed_from_url_and_requires_confirmation	164-177	Checks verification link is removed from url and requires confirmation.
test_email_reset_screen_revokes_sessions	179-198	Checks email reset screen revokes sessions.
test_email_enabled_registration_opens_home_before_verification	200-213	Checks email enabled registration opens home before verification.

The final `__main__` guard calls `unittest.main()` when this file is run directly. Discovery can also load it as part of the full suite.

tests/test_database_setup.py

7 setup tests: mocked input/clipboard/connections verify URI/TLS handling and that failure never prints credentials or saves new settings.

DatabaseSetupTests (lines 10-81): A `unittest.TestCase` group; assertions report success or a precise failure.

Test function	Lines	Purpose
test_connection_diagnostics_hide_provider_details_and_credentials	11-22	Checks connection diagnostics hide provider details and credentials.
test_clipboard_failure_is_private_and_does_not_change_settings	24-32	Checks clipboard failure is private and does not change settings.
test_clipboard_input_reaches_validation_without_being_printed	34-43	Checks clipboard input reaches validation without being printed.
test_provider_uri_preserves_password_and_tls_parameters	45-51	Checks provider uri preserves password and tls parameters.
test_rejects_insecure_incomplete_and_non_postgres_urls	53-61	Checks rejects insecure incomplete and non postgres urls.
test_invalid_input_does_not_echo_credentials	63-70	Checks invalid input does not echo credentials.

Test function	Lines	Purpose
test_connection_failure_does_not_echo_credentials_or_write_settings	72-81	Checks connection failure does not echo credentials or write settings.

tests/test_email_actions.py

9 token tests: single-use/expiry/purpose rules, eligibility, cooldown, account changes, reset revocation and simultaneous claims.

EmailActionTests (lines 14-130): A unittest.TestCase group; assertions report success or a precise failure.

EmailActionTests.setUp | lines 15-18

Creates the fixture used by each test, such as a disposable database/account or a small purchase list. This runs before each test so unrelated checks do not share state.

EmailActionTests.tearDown | lines 20-22

Disposes database engines, clears relevant cached services/environment patches and removes temporary files after each test. This keeps real accounts separate from test evidence.

EmailActionTests.allow_request | lines 24-26

Clears the resend cooldown in the disposable database so a test can request another email action immediately without waiting.

EmailActionTests.verify | lines 28-31

Requests and consumes a fixture verification token, then clears its resend cooldown to prepare a verified test account.

EmailActionTests.test_simultaneous_reset_claims_only_one_token.consume | lines 114-119

A nested concurrency-test helper tries to claim a reset token and returns whether it succeeded. Two workers prove only one can consume it.

Test function	Lines	Purpose
test_verification_gates_login_stores_hash_and_is_single_use	33-45	Checks verification gates login stores hash and is single use.
test_expired_and_wrong_purpose_links_cannot_verify	47-56	Checks expired and wrong purpose links cannot verify.
test_reset_request_requires_verified_email_and_unknown_is_silent	58-62	Checks reset request requires verified email and unknown is silent.
test_resend_is_limited_and_new_request_invalidates_old_link	64-71	Checks resend is limited and new request invalidates old link.
test_reset_revokes_sessions_rotates_recovery_and_rejects_replay	73-88	Checks reset revokes sessions rotates recovery and rejects replay.
test_email_change_invalidates_old_links_and_requires_reverification	90-102	Checks email change invalidates old links and requires reverification.
test_recovery_code_reset_invalidates_email_reset_links	104-109	Checks recovery code reset invalidates email reset links.
test_simultaneous_reset_claims_only_one_token	111-122	Checks simultaneous reset claims only one token.
test_account_deletion_removes_email_actions	124-130	Checks account deletion removes email actions.

tests/test_email_delivery.py

6 SMTP/link tests: secure transport order, trusted addresses, hidden settings and silent ineligible requests.

DeliveryTests (lines 7-58): A unittest.TestCase group; assertions report success or a precise failure.

DeliveryTests.mailer | lines 8-13

Builds a Mailer with fictional settings and optional overrides. Transport methods are mocked, so this fixture does not contact a real email provider.

Test function	Lines	Purpose
test_rejects_untrusted_link_addresses_and_insecure_transport	15-21	Checks rejects untrusted link addresses and insecure transport.
test_representation_does_not_contain_credentials	23-24	Checks representation does not contain credentials.
test_link_and_expiry_use_trusted_config_and_correct_purpose	26-33	Checks link and expiry use trusted config and correct purpose.
test_ssl_transport_authenticates_without_plaintext_connection	35-40	Checks ssl transport authenticates without plaintext connection.
test_starttls_happens_before_login	42-47	Checks starttls happens before login.
test_unknown_address_never_sends_and_worker_errors_do_not_escape	49-58	Checks unknown address never sends and worker errors do not escape.

tests/test_insights.py

7 trend/filter tests: year boundaries, exact totals/rounding, missing versus zero-spend baselines, combined filters and numeric sorting.

InsightsTests (lines 7-54): A unittest.TestCase group; assertions report success or a precise failure.

InsightsTests.setUp | lines 8-11

Creates the fixture used by each test, such as a disposable database/account or a small purchase list. This runs before each test so unrelated checks do not share state.

Test function	Lines	Purpose
test_months_cross_year_boundary	13-16	Checks months cross year boundary.
test_trend_keeps_empty_months_and_uses_exact_totals	18-22	Checks trend keeps empty months and uses exact totals.
test_insights_average_balance_and_previous_calendar_month	24-29	Checks insights average balance and previous calendar month.
test_empty_or_missing_baseline_is_not_presented_as_a_change	31-36	Checks empty or missing baseline is not presented as a change.
test_free_prior_purchase_is_a_valid_baseline	38-41	Checks free prior purchase is a valid baseline.
test_filters_combine_without_mutating_records	43-48	Checks filters combine without mutating records.

Test function	Lines	Purpose
test_price_and_impact_sorts_use_numbers	50-54	Checks price and impact sorts use numbers.

The final `__main__` guard calls `unittest.main()` when this file is run directly. Discovery can also load it as part of the full suite.

tests/test_logic.py

20 rule/backup tests: Decimal estimates, validation, month grouping, badge boundaries, sample data, factor provenance and safe import/export.

LogicTests (lines 9-121): A `unittest.TestCase` group; assertions report success or a precise failure.

LogicTests.purchase | lines 10-11

Returns a valid synthetic purchase through `make_purchase` with overridable `category/price/date`, providing short fixtures for rule tests.

Test function	Lines	Purpose
test_formula_and_decimal_rounding	13-15	Checks formula and decimal rounding.
test_invalid_prices	17-20	Checks invalid prices.
test_empty_brand_and_invalid_dates	22-25	Checks empty brand and invalid dates.
test_unknown_category	27-29	Checks unknown category.
test_zero_price_allowed	31-32	Checks zero price allowed.
test_year_and_month_filter	34-37	Checks year and month filter.
test_summary_known_values	39-41	Checks summary known values.
test_empty_no_badges	43-44	Checks empty no badges.
test_eco_saver_boundary	46-52	Checks eco saver boundary.
test_lower_factor_badge_boundary	54-58	Checks lower factor badge boundary.
test_edit_recalculates_and_keeps_id	60-66	Checks edit recalculates and keeps id.
test_demo_has_15_and_handles_january	68-72	Checks demo has 15 and handles january.
test_all_categories_have_alternatives	74-79	Checks all categories have alternatives.
test_backup_roundtrip	81-83	Checks backup roundtrip.
test_backup_does_not_trust_estimate	85-88	Checks backup does not trust estimate.
test_legacy_backup_requires_explicit_migration	90-95	Checks legacy backup requires explicit migration.
test_factor_provenance_and_no_invented_reuse_discount	97-106	Checks factor provenance and no invented reuse discount.
test_duplicate_ids_rejected	108-111	Checks duplicate ids rejected.
test_malformed_backup	113-116	Checks malformed backup.

Test function	Lines	Purpose
test_csv_formula_neutralized	118-121	Checks csv formula neutralized.

The final `__main__` guard calls `unittest.main()` when this file is run directly. Discovery can also load it as part of the full suite.

How the assertions work

`assertEqual/assertTrue/assertFalse` compare expected results with actual output. `assertRaises/Regex` check deliberate failure paths. `subTest` repeats a rule across inputs. `patch` and `Mock` replace SMTP, clipboard or database calls for isolated checks; they are test tools, not production account logic. `TemporaryDirectory` contains disposable files. `ThreadPoolExecutor` exercises parallel token claims, and test-specific hooks create reproducible race conditions.

12. Configuration, dependencies and references

File / component	Explanation
requirements.txt	Pins Streamlit 1.55.0 for the UI, SQLAlchemy 2.0.43 for engines/SQL, psycopg[binary] 3.2.10 for PostgreSQL and argon2-cffi 25.1.0 for passwords. Python's json/csv/datetime/decimal/turtle/sqlite support is standard-library functionality. The tools' toml import is available with this environment's Streamlit dependencies.
.streamlit/config.toml	Sets green primary colour, cream background, pale secondary surfaces, dark text and sans-serif font. <code>server.maxUploadSize=2</code> limits uploaded files to 2 MB; <code>storage.py</code> also validates backup size.
.streamlit/secrets.toml	Private database and optional SMTP configuration used at runtime. Values are not reproduced. This file and private temporary files are excluded from Git.
.vscode/tasks.json	Three process tasks use the project virtual environment to install dependencies, start Streamlit or run unittest discovery from the workspace folder.
.vscode/launch.json	The debugpy launch configuration runs the streamlit module with <code>app.py</code> , project working directory, integrated terminal and justMyCode enabled.
.github/workflows/tests.yml	On a future repository push/pull request, Ubuntu checks out the code, installs Python 3.12/dependencies and runs unittest discovery. The workflow exists, but a live GitHub run is not claimed here.
.gitignore	Excludes the virtual environment, Python caches, private secrets/temp files and <code>personal_data</code> . Generated public Turtle artwork and synthetic test data can be shared.
README.md and docs/	Explain running, features, design, methodology, tests, Neon/Gmail setup, deployment, requirement status and a genuine-feedback template. Screenshots show synthetic QA data.

Run and inspect the code

```

.\.env\Scripts\python.exe -m streamlit run app.py
.\.env\Scripts\python.exe -m unittest discover -s tests -v
.\.env\Scripts\python.exe tools/draw_rewards.py

```

Open the entire SA PYTHON folder in VS Code. Use the function/block line references in this guide to find the relevant code. A virtual environment must exist before those commands work; another computer needs Python 3.12,

environment creation and requirements installation first. Configuration tools and account operations can create or change data, so use disposable settings when experimenting.

Sources and attribution

EPA source catalogue: dataset definition and download

EPA recorded CSV: source factor file

World Bank: recorded 2022 INR/USD API request

Python standard library: Decimal

Python standard library: Turtle

Streamlit: database connections

Numerical factors, currency value, source rows and brand-option descriptions are explained from the project's bundled data snapshot. The EPA catalogue was also checked while preparing this guide. Alternative links are recorded as checked on 2 October in the app; availability and provider terms may change. This guide does not claim live prices, certified footprints or new independent brand research.

Code coverage of this guide

Coverage checked against Python's syntax tree: 20 authored Python files, 178 named class/function/method units, all 78 test functions, the three files with module-level UI code, four data JSON files, SVG integration and the public configuration files. Setup pages and app.py are explained as executable blocks; generated and installed code is not presented as student-authored work.

This PDF documents code and behaviour. It is not evidence of completed public hosting, Gmail delivery, human feedback or a guaranteed assessment grade.